



Summary of The Lean Startup

Original book by Eric Ries

The *Lean Startup* by Eric Ries is considered a bible in the tech entrepreneurship community. It's a methodology for creating businesses that focuses you on finding out what customers actually want. It uses concepts of scientific experimentation to prove that you're making progress. It encourages you to launch as early and cheaply as possible so you don't waste time and money.

Learn the critical concepts of the Minimum Viable Product, cohort metrics, A/B testing, virality, and startup pivots.

This document is restricted to the personal use of the individual owner of the email account rakib.py+free@gmail.com (the "Subscriber"). No part of this document may be reproduced or distributed by any means other than for personal use of the Subscriber, nor shared with individuals other than the Subscriber, without prior written permission of Shortform Inc.

1-Page Summary

The goal of a startup is to learn what their customers want and will pay for, as quickly as possible.

Because startups face so much uncertainty, you have to make continuous adjustments to your startup plan, based on the information you get back. This is Build-Measure-Learn.

Think about it like driving a car. When you get on the road, you make constant adjustments to the steering wheel, based on where you see yourself on the road. Veer a little right, and you turn to the left before you go offroad.

A startup's the same way. You collect information about your customer, just like seeing where you are on the road. Based on this new information, you adjust your strategy, like turning the steering wheel. You keep repeating this and making progress.

But you need to approach learning with discipline, a process called "validated learning."

Bad learning is executing a plan without much prior thought, seeing it work or fail, and giving a post-hoc rationalization. ("Well of course X didn't work, that means we should do Y!")

Validated learning is having testable hypotheses about the world, designing experiments to test those hypotheses, and analyzing the data to evaluate your hypotheses. You have real, quantitative data to show what you have learned.

How can you learn what your users want, as quickly as possible, and with as few resources as possible? The strategies of Lean Startup answer this question.

Build-Measure Learn

First, you set your hypothesis. What do you believe about your customers that is vital to your business? How are you going to measure this?

Next, you **build the MINIMAL product necessary to test the hypothesis.**

Next, you **run the experiment.** Often this means exposing users to the product and collecting data on their behavior.

Finally, you **analyze the data and reflect** – how far off was your hypothesis? What do you need to change about your strategy? Should you actually change your entire direction?

Then you update the hypothesis or set a new one. Then you build the minimal product to test that hypothesis, and so on.

The faster you move through this loop, the faster you'll learn, and the more progress you'll make. Imagine how much you'd learn from 10 steadily improving prototypes vs 1 giant, fully-featured prototype.

Startup Hypotheses

What are hypotheses in a startup? Your hypotheses should revolve around the most important problem of a startup – how to build a sustainable business around your vision.

Commonly, there are two critical hypotheses:

Value hypothesis: does the customer have the problem you're trying to solve? Does the product actually deliver value to the customer?

Growth hypothesis: how will the company grow once people start using the product?

Principles of Lean Experimentation

The best way to understand human preferences is to track behavior of real people, not ask for opinions. People often can't verbalize what they actually want – cue the classic Henry Ford quote, "If you asked people what they wanted, they'd have said faster horses." The better way to measure what people want is by their real behavior - what they click on, what they spend time looking at, what they choose to pay for.

Think about the cheapest, fastest experiment you can run to validate the hypothesis. Simplify the product to the core essentials needed to run the experiment. Resist the urge to build more than is absolutely necessary – as you'll learn, sometimes you can fake it 'til you make it without a real product.

Launching early gives you customer information earlier. The earlier you learn if customers actually want what you're building, the more time you have to change your plan and run more experiments. You also discover customer concerns you couldn't have predicted in a vacuum.

The Minimum Viable Product

Your goal is to move through the Build-Measure-Learn loop as quickly as possible. Even though the loop has 3 steps, Build is often the step where you will waste the most time.

The critical question you need to answer is: what is the **MINIMUM product you can build to get reliable data on your hypothesis?**

This product is termed the Minimum Viable Product (MVP) and is one of the most important concepts in Lean Startup. This is the product that is the bare minimum to test your hypothesis. Unlike normal product development, **you are NOT aiming for product perfection** – you're merely trying to start learning as soon as possible.

Depending on your business, there are a variety of ways to test your hypothesis in the cheapest, fastest way possible. In the more extreme, you don't need a product at all - you can simply pretend you have a product and collect user behavior.

Here are examples of MVP types:

- **Landing Page MVP:** create a web page describing the features of the product, before a product actually exists. Track clicks and attempted downloads to gauge how much users want it.
- **Video MVP:** make a video simulating what the product does. Even without using the actual product, watching the video will give enough info for viewers to decide whether they want the product shown.
- **Concierge MVP:** instead of building processes that scale, run very manual processes dependent on special white-glove treatment. Often the founders themselves deliver the service. This accelerates learning and allows quick iteration on the product.
- **Wizard of Oz MVP:** if you plan to build fancy automated technology, try to test it with a human behind the scenes. The human performs the service that the technology does, and the user is none the wiser.

Choosing the Right Metrics

Defining the right metrics that actually matter to your business is critical.

The most insidious kind of metrics, vanity metrics, give you false optimism – it *seems* like you're making great progress, but in reality you're actually stuck. Often, **vanity metrics are metrics that have no choice but to keep increasing over time**. One common example is total user count. Let's say your app adds 1,000 users every week. At the end of 10 weeks of hard work, you have 10,000 users. This is a big number!

Except you aren't growing any faster – you're still adding 1,000 users every week. Your hard work actually didn't change your *growth rate*.

Instead of looking at cumulative vanity metrics over time, **the more accurate analysis is to separate users into groups based on the time they joined, then measure your metric for each group independently**. Each group of users is called a **cohort**. For instance, you can measure the signup rate for each week separately to see if that's increasing over time.

A/B Testing

Let's say you develop a new feature to your product and you release it to all your users. Suddenly your metrics improve. But how do you know seasonal effects aren't at play – that the users who joined later aren't just naturally more engaged? Or that you got a burst of users from an unexpected news article?

An A/B test avoids bias by splitting users into seeing two different versions of your product. By analyzing the metric resulting from both groups, you get strong quantitative evidence about which version users like more.

For example, let's say you have a landing page MVP listing your potential features and a signup form. You're not sure which of two features your users will like more. So you set up an A/B test – half of visitors see feature A on your landing page; the other half sees feature B. You measure the difference in signup rate. If feature A gets a 5% signup rate, but feature B gets 2%, this is evidence that your users may prefer feature A!

Another benefit of A/B testing is that **it lowers politics**. You don't have to squabble with your team over which features are better – you can put it to the test with an MVP. A/B testing also lets you assign credit where it's due.

Pivoting Your Startup

Now when you're facing the data, you need to decide what to do. Is there still promise in your direction, and should you keep trying to iterate through the Build-Measure-Learn loop? Or have the metrics come back so disappointing so often that it's time to change your strategy entirely – to pivot?

The answer is often unclear because you will seldom encounter complete, abject failure. The more likely state is when you're barely limping along, not plummeting to the earth but also feeling like you're not really making progress. The decision gets hard here.

There are two signs of the need to pivot:

- Your metrics are not good enough to meet your goals for your startup.
- Your experiments are leading to less and less progress (which is a sign that you're out of good ideas)

To make sure your startup is on the right track, have a regular “pivot or persevere” meeting. The frequency should be between once every few weeks and once every few months, depending on your startup - the more volatile your startup and the less time you have, the more frequent this should be.

Pivots come in multiple flavors. Which type you make depends on what you learn in your experiments about customer needs.

Pivot types include:

- **Zoom-in pivot:** Customers like a specific feature of your product. You pivot so your product now focuses entirely on delivering this feature.
- **Customer need pivot:** You know your customers well, and the problem you’re solving for them is not very important. You pivot to target a new customer need, which may entail an entirely new product.
- **Business architecture pivot:** You pivot to change between two types of businesses: high margin and low volume, or low margin and high volume. This roughly matches to a business to business (B2B) model, or a business to consumer (B2C) model.
- **Technology pivot:** You want to solve the same problem for the same users, but you pivot to use a different technology to do so. Often, this offers better price or performance.

Introduction

The media portrays startup success incorrectly as fatalistic – if you have the right stuff (a good idea, determination, timing, and luck), you will inevitably succeed. If you don't succeed, it's because you didn't have the right stuff. You either have it or you don't.

This idea is seductive because it both promises easy success and justifies failure. To succeed, all you need is the right stuff – easy! And yet if you fail, you can simply justify failure as not having the right stuff, rather than making poor decisions. This is softer on the ego.

This is the wrong way to think about entrepreneurship. Startup success is NOT fatalistic. **There is a rigorous, repeatable method to achieve startup success – the Lean Startup.**

Origins of the Lean Startup

The ideas in the book came about when Eric Ries got frustrated working hard on products that failed to get traction. As an engineer, he initially thought they failed due to technical problems, but this was never the right answer. **In reality, they just wasted a lot of time building things nobody wanted.**

So when he started his new company, IMVU, he wanted to try something different. One inspiration was Steve Blank's idea of Customer Development: a rigorous methodology for the business and marketing side of a startup. Another inspiration was Japan's lean manufacturing systems, made famous by Toyota.

He applied these concepts to IMVU, which became a roaring success, with millions of users and \$50 million in annual revenues in 2011. To help others succeed in innovation, Eric started the Lean Startup movement by publicizing the framework you'll learn about here.

The 5 Principles of Lean Startup

There are 5 main concepts in the Lean Startup. We'll be exploring each one in much more detail.

1) Entrepreneurs are everywhere. Eric defines a startup as “a human institution designed to create new products and services under conditions of extreme uncertainty.” This is purposely inclusive and covers a range of people, from employees in a large company or the government to the stereotypical college dropouts in a garage.

2) Entrepreneurship is management. Management shouldn't be a dirty word in startups. Instead of a chaotic, “do what we feel like” strategy, you need to adopt a principled approach to manage risk and reduce failure.

3) Validated Learning. The job of a startup is to learn who its customer is and what its product should be. This learning should be treated rigorously and scientifically.

4) Build-Measure-Learn (and repeat). First you build, then you measure the results, then you learn what to improve next time. Then you build again. By stepping through this loop, you'll gain concrete information on your hypotheses about the world and decide whether to change your strategy. The faster you iterate through this loop, the more you'll learn and the more progress you'll make.

5) Innovation accounting. It's critical to treat learning rigorously, which means measuring progress and

creating action plans.

Overview of the Book

This book is organized into three sections:

- Vision: We define what an entrepreneur is and how startups learn through experimentation.
- Steer: We step through the Build-Measure-Learn loop in technical detail, covering concepts like the Minimum Viable Product (MVP) and A/B testing.
- Accelerate: We learn how to step through the Build-Measure-Learn loop faster and smarter.

Part 1: Vision | Chapter 1: Startups Need Managing

'Management' is often a dirty word in startups because it conjures images of grey suits, bureaucracy, and TPS reports. Startups are worried management will squash energy and creativity.

The problem is, startups go too far in the other direction into chaos. They often take a shoot-from-the-hip, hail-Mary, undisciplined approach to company development. This unfortunately often leads to failure, spending years of your life building something no one cares about or remembers.

Startups really *do* need management, but a new kind of management catering to high-risk innovation - a kind of Entrepreneurial Management. By using this management method, you will be confident you're moving in the right direction and learning more about your company. Surprise – that method is the Lean Startup.

Startups Need to Learn as Quickly as Possible

The goal of a startup is to learn what their customers want and will pay for, as quickly as possible.

Thus the real metric you should care about is the amount you learn – not the number of hours you work, the lines of code you've written, or the number of times you've banged your head against the wall.

Because startups face so much uncertainty, you have to make continuous adjustments to your startup plan, based on the information you get back. This is Build-Measure-Learn.

Think about it like driving a car. When you get on the road, you make constant adjustments to the steering wheel, based on where you see yourself on the road. Veer a little right, and you turn to the left before you go offroad.

A startup's the same way. You collect information about your customer, just like seeing where you are on the road. Based on this new information, you adjust your strategy, like turning the steering wheel. You keep repeating this and making progress.

Unfortunately, some startups avoid this learning cycle. Instead, they essentially point themselves in one direction, put on a blindfold, and then slam their foot on the pedal. To no one's surprise, they end up in a ditch by the side of the road.

(Shortform example: The failed online grocery company Webvan in the 2000 dot-com bubble is a classic example of this – before fully validating their customer, they spent over a billion dollars building out their infrastructure and delivery fleet. To their chagrin, [there weren't enough customers to justify the investment](#), and the company folded.)

Chapter 2: Entrepreneurs Are Everywhere

Who, exactly, is an entrepreneur?

Entrepreneurs aren't just startup founders working out of apartments. They can also be general managers in large companies charged with creating new ventures or new product lines.

The author **defines a startup as “a human institution designed to create a new product or service under conditions of extreme uncertainty.”**

The definition is purposefully broad – it can apply to a non-profit, a government agency, a big company or a small company. The key piece of this definition that makes a startup unique is the condition of “extreme uncertainty.”

In situations of uncertainty, traditional management tools – like forecasts, business plans, and milestones – break down. There's too much that's unknown about the world to predict with high accuracy what's going to work.

The Lean Startup is a set of methods for building a successful startup, wherever it is and whoever is working on it.

Startup Example: Snaptax

In 2009, a startup wanted to automate W-2 form processing to streamline individual taxes. Their early users had difficulty scanning in their tax forms, so instead the startup decided to switch to cell phone cameras as a way to capture W-2 forms. But the customers asked for something even more ambitious – could they complete their entire tax return on the phone?

This was a tall order. Tax forms can get super complex and annoying to deal with.

Instead of building a complete product and shipping a giant package, the startup decided to release a barebones version. It only processed the simple 1040EZ tax return, and it only worked for California. It launched as SnapTax in 2011 to great success.

So who was behind SnapTax? Surprise! It was an internal project at Intuit, a giant public company that makes finance tools like Quicken and TurboTax.

Intuit: a Giant Startup

Intuit was founded by Scott Cook in 1983, and they dominated the finance and tax prep software industries. But by 2002, its product initiatives were failing. Cook realized the management practices at Intuit couldn't keep up with the rapidly changing economy.

A decade later, Intuit has built entrepreneurship and risk taking into the backbone of their company. For example, TurboTax, one of their flagship products, used to run on an annual product improvement cycle, where product and marketing teams would package together the year's changes and push it out in a single big release.

Nowadays, they move in a much more agile way – they'll run up to 70 different tests on one week, examine the data the next week, and quickly decide what further tests they need to run. Furthermore, because they rely on data to make decisions, good ideas win, rather than politics. This has led to much faster growth of

new product lines.

Eric's major point is that **innovation can happen anywhere**, not just in college dorm rooms but also in large, experienced organizations. But in the latter, it's up to the leadership to create the conditions that will stimulate entrepreneurship and experimentation.

Chapter 3: Learn What Your Users Want

Remember that a startup is trying to build a new product or service in conditions of extreme uncertainty. There are a lot of things you know that you don't know, and a lot of things you don't know that you don't know.

Therefore, **a startup's most important function is learning** – in particular, learning what the customers really want and what will lead to a sustainable business.

But you need to approach learning with discipline, a process called “validated learning.”

Bad learning is executing a plan without much prior thought, seeing it work or fail, and giving a post-hoc rationalization. (“Well of course X didn't work, that means we should do Y!”)

Validated learning is having testable hypotheses about the world, designing experiments to test those hypotheses, and analyzing the data to evaluate your hypotheses. You have real, quantitative data to show what you have learned.

Startup Example: IMVU

In 2004, author Eric Ries and his co-founders at IMVU wanted to build a social network around instant messaging (IM), which seemed attractive for its network effects – the more people who join, the more valuable the network is, which makes even more people join.

Because of network effects, the top IM products owned the vast majority of the industry. It was commonly accepted, almost obvious, that it'd be extremely difficult to make a new IM network succeed. If you're already on a network with all your friends, you don't want to switch to a new network without all your friends joining too. (Shortform note: think about how hard it'd be to build a rival to Facebook today – how would you get Facebook users to join your network when all their friends are on Facebook? Would they really want to join yet another social network?)

To avoid competing with incumbents head-on, IMVU decided not to start their own new IM network. Instead, they built a new 3D video game layer on **top** of IM networks. Users would first join IMVU, a 3D virtual world where they could create avatars and buy virtual goods. Then they would connect their IM accounts and message people on their existing IM networks to join IMVU too. Critically, this meant users wouldn't have to switch to a whole new IM network – it already works with the IM networks they belong to. Ingenious!

The IMVU team labored for 6 months on their prototype product. They worried constantly about the details of their implementation – “how many IM networks should we support?” (over a dozen) “How buggy can the prototype be? Will it make us look bad?”

Finally, with their pride on the line, they launched the product.

And no one joined.

They thought it was a quality problem at first, so they worked on fixing bugs and adding features. This didn't budge the needle.

Finally, **they decided to bring in potential users for interviews**. This is where their epiphany happened.

IMVU had built the whole game around getting new users to bring in their friends from other IM networks. But users didn't actually *want* to invite their friends over before they had a chance to really test it out – if the game was uncool, they'd look bad.

Even more importantly, **IMVU found that users actually *didn't mind* joining a new IM network**. Just like people today have different lives on different social networks (Facebook, Twitter, LinkedIn, Instagram, Pinterest, etc.), IMVU found that their target customer wanted a separate IM network dedicated to this new virtual world. They wanted to make new friends, not bring their existing friends over.

When Eric and his team realized this, they realized their product direction was totally misguided and counter to what their users wanted. This forced them to throw away almost a year of hard work, which was painful. But it needed to happen. When IMVU started building a product that their customers actually wanted, they started getting traction.

How to Learn Earlier and Faster

This story represents the critical lesson of Lean Startup – if you don't understand what your customer actually wants, you'll waste a lot of time building something no one will buy. This will be painful, and the more time you spend on this, the harder it'll be to throw it all away.

Put more proactively – **how could IMVU have learned what customers wanted earlier**, without 6 months of hard engineering? Did they really need to support a dozen IM clients at launch? Probably not – even connecting just **one** IM network could have led to the same conclusion, and it would have taken a lot less work.

But what if they had tried something even more extreme – what if they *pretended* to have a working product on their website with a download button, and they measured what features users actually wanted by download clicks?

This question haunted him for a while. What if there were a way to avoid wasting months, years of work while also making significantly more progress in the business?

Eric's experience running IMVU was the inspiration for Lean Startup. The rest of the book teaches a method for you to avoid what IMVU did.

Chapter 4: Experiment like a Scientist

Lean Startup treats building a startup as science:

1. Create a hypothesis. What do you believe that's important to your business?
2. Design an experiment to directly test that hypothesis. Run the experiment and gather data.
3. Reflect on the data – can you validate or reject your hypothesis?

Once again, this stands in stark contrast to the typical “launch and we’ll see” approach. You’re forced to question what your assumptions about your business are, and you’re encouraged to test those assumptions as quickly and cheaply as possible.

Startup Hypotheses

What are hypotheses in a startup? Your hypotheses should revolve around the most important problem of a startup – how to build a sustainable business around your vision.

Commonly, there are two critical hypotheses:

Value hypothesis: does the customer have the problem you’re trying to solve? Does the product actually deliver value to the customer?

Growth hypothesis: how will the company grow once people start using the product?

We’ll unpack each of these, and how to test them, in the following chapters.

Principles of Lean Experimentation

As you learn how to design experiments and test hypotheses, here are important principles to remember throughout.

The best way to understand human preferences is to track behavior of real people, not ask for opinions. People often can’t verbalize what they actually want – cue the classic Henry Ford quote, “If you asked people what they wanted, they’d have said faster horses.” The better way to measure what people want is by their real behavior - what they click on, what they spend time looking at, what they choose to pay for. When someone behaves this way, you know they really want it.

Think about the cheapest, fastest experiment you can run to validate the hypothesis. Simplify the product to the core essentials needed to run the experiment. Resist the urge to build more than is absolutely necessary – as you’ll learn, sometimes you can fake it ‘til you make it without a real product.

Launching early gives you customer information earlier. The earlier you learn if customers actually want what you’re building, the more time you have to change your plan and run more experiments. You also discover customer concerns you couldn’t have predicted in a vacuum.

All these principles are in strict contrast to the usual market research/strategic planning process. Traditionally, you would try to research everything possible about your core user, then build your product to polished perfection, then release with a big launch party. This invites failure when you build something customers actually don’t even want.

We’re going to run through a few examples showing these principles at work. Notice how the same

underlying principles apply to vastly different companies and scenarios.

Startup Example: Zappos

During the dot-com boom, it seemed like the internet could be a new commerce platform for everything – books, groceries, even pet supplies. Companies like Webvan started business by building massive infrastructures and supply chains, even before they had proven customer demand.

Zappos founder Nick Swinmurn took the opposite approach. **His first action was to test the hypothesis that people actually wanted to buy shoes online.** He took perhaps the simplest approach possible – he went to local shoe stores, took pictures of shoes, and posted them online. If a customer actually bought a shoe, he'd buy the shoe and ship it himself.

Isn't this brilliant? **With virtually zero start-up cost, Zappos tested customer demand.** If you asked someone how they'd start an online shoe company, a likely answer would be to buy the most popular shoes to sell. But Zappos skipped all that. They didn't need to stock warehouses full of inventory and manage supply chain issues. This would have been a waste – they didn't even know if people wanted to buy shoes.

Furthermore, they measured customer demand in the best way possible –getting people to pay real money. This is a much stronger indication of interest than voicing an opinion on a survey.

From this starting point, Zappos could easily run all sorts of deeper experiments – what types of shoes do people want? What policies matter most to them? By measuring what people actually do, Zappos can learn true customer preferences.

Startup Example: Village Laundry Service

In India, washing machines are scarce. Laundry is primarily done at home or by paid cleaners who wash clothes in river water and hang to dry, often taking over a week to return clothes.

Village Laundry Services saw an opportunity to introduce laundry services with modern machines for people who couldn't afford them. Their hypothesis? Customers would pay to have their laundry cleaned and returned within the same day.

To experiment, **they launched a simple prototype** – a truck with a washing machine mounted on the back. This was just a marketing gimmick – they really took the laundry somewhere offsite and cleaned it there. But it was just enough to test their hypothesis with real customers and find out what they really wanted.

For example, they learned that the truck was actually kind of sketchy looking – customers were afraid they'd drive away with their clothes, never to return. In response, they built a more substantial, reputable-looking kiosk. They also found that customers were willing to pay a premium for faster service and ironing.

Through all their learning, Village Laundry Services ended with their final version – a mobile kiosk with its own washer and dryer that could service clothes on site.

These two examples seem vastly different on the surface. One was leading the forefront of e-commerce during the dotcom boom. Another was introducing hygienic convenience to a developing region.

But the principles were the same:

- They formed a hypothesis about the core customer need that would drive their business.
- They started with the bare minimum product needed to test the hypothesis.
- They studied real user behavior by observing their actions, not simply by asking questions.
- From this baseline, they could experiment further.

No matter your situation or customer need, these principles will apply to you as well.

Exercise: Your Startup Beginning

Reflect on the principles of Lean Startup.

Do you consider yourself in an entrepreneurial situation – “creating a new product or service under conditions of extreme uncertainty?” Why or why not?

Have you ever felt that you waited too long to launch a new product, service, or project? What prevented you from launching earlier? What could you have gained had you launched earlier?

In your current project, what do you take for granted that you should actually question? How can you test this in the next week?

Part 2: Steer | Chapter 5: Form Your Hypothesis

With your startup, your goal is to learn as quickly as you can with disciplined experiments. Central to Lean Startup is the **Build-Measure-Learn feedback loop**.

Let's step through this loop once:

First, you set your hypothesis. What do you believe about your customers that is vital to your business? How are you going to measure this?

Next, you **build the MINIMAL product necessary to test the hypothesis**.

Next, you **run the experiment**. Often this means exposing users to the product and collecting data on their behavior.

Finally, you **analyze the data and reflect** – how far off was your hypothesis? What do you need to change about your strategy? Should you actually change your entire direction?

Then you update the hypothesis or set a new one. Then you build the minimal product to test that hypothesis, and so on.

The faster you move through this loop, the faster you'll learn, and the more progress you'll make. Imagine how much you'd learn from 10 steadily improving prototypes vs 1 giant, fully-featured prototype.

Startup Hypotheses

Every startup begins with a set of hypotheses about how the business will work. As stated above, there are typically two core hypotheses: the Value hypothesis (what your users want), and the Growth hypothesis (how your startup will grow).

Some hypotheses are very un-risky – they're very likely to be true. If your business will rely on advertising for revenue, for example, chances are good that you'll be able to procure the same advertising rates as competitors in your industry. Online advertising has matured enough that advertisers know how much they want to pay for which types of ads across the Internet.

But other hypotheses are much riskier and less likely to be true. Often, these riskiest hypotheses underlie the reason your business exists. Namely:

- Do people actually have the problem you believe they have?
- Do they actually want what you're offering?
- Are they willing to pay for it?

The Dangerous Startup by Analogy

One insidious way to frame your business is by **analogy to other companies**. If there's a successful company that succeeded because of attribute X, then it's tempting to say that because you also have attribute X, you'll naturally succeed.

This superficially looks like sound logic, but in reality **it's very weak and papers over a lot of assumptions** that may not hold up on closer inspection.

(Shortform example: for example, when Uber paved the way for on-demand services, a litany of companies cropped up to start the “Uber of [blank].” The logic for an on-demand service was as follows:

We are the Uber of laundry servicing. Uber allows users to request a car on demand using a mobile app, reducing friction far beyond hailing a taxi. Similarly, our users request laundry services on demand, we pick the clothes up and launder in our laundry rooms, and we deliver their laundry to their home 24 hours later. The laundry industry is worth \$15 billion. Just as Uber is now larger than the entire taxi industry, we will be larger than the entire laundry industry.

Sound good?

This argument sounds superficially reasonable. In particular, by merely associating with an uber-successful company, your idea looks better.

But this analogy **hides the underlying assumptions** about how the business will work. A more accurate restatement of the pitch is as follows:

Lots of people who don't have cars need transportation on demand. Before Uber, getting a car was a huge pain. Hailing a taxi took minutes of standing in the rain, and you might need to call ahead by 30 minutes to get a taxi to drop by. Uber built a mobile app that allows riders to request a car frictionlessly. To service riders, Uber accesses cheap labor through part-time contractors who use their own vehicles (fully paid for by the contractor). This lowers the cost of each ride, creating a virtuous cycle wherein more riders led to more drivers joining, leading to lower costs and faster pickups, leading to more riders. They're now bigger than the taxi industry because cheaper rides and lower friction encouraged more usage – I certainly use Uber now more than I ever used taxis.

We'll do the same for the laundry industry. The friction for laundry is currently high – you either go to a Laundromat, or you have to trudge your laundry across your home to your laundry machines. Then you have to move your clothes to the dryer and fold your clothes. The average laundry load costs \$2 at a Laundromat, or \$0.50 at home. That's where we come in. Users can request laundry services in real time. We pick up the laundry and process them in our laundry facilities. We have economies of scale that lower costs below what users pay now. In fact, users will be willing to pay above their cost of laundry because of the time saved. Like Uber, we'll grow to be larger than the current laundry industry because when users realize how convenient our service is, they'll do laundry more than they used to.

This more insightful explanation has key assumptions written out. And suddenly the business looks more shaky. First, there are now facts about Uber to confront. Is this really why Uber has succeeded winning? Are they the right company to emulate?

More importantly, there are now a lot of assumptions about the on-demand laundry business to question. Is doing laundry so painful that people would really pay a premium to save the time? Is it possible to get pickup and delivery costs under what users are willing to pay? Will people really do more laundry than they used to, when it's more convenient?)

Beware of when you compare yourself to successful companies, and on that basis alone believe you'll succeed. You're likely making hidden assumptions that need to be validated.

Even before you build a prototype, it's useful to validate these assumptions, if only to get a quick sanity check. You can do this by talking directly to your target customers.

Talk to Customers and See for Yourself

Once again, here are the key questions to answer about your product or service:

- Do people actually have the problem you believe they have?
- Do they actually want what you're offering?
- Are they willing to pay for it?

In the last chapter, we discussed how to *robustly* answer these questions – build a minimal prototype and study real customer behavior using the Build-Measure-Learn loop. We know that real customer actions are the best form of data.

But when you're starting out, it's often worthwhile to **do a sanity check by spending time with potential customers**. In Japanese manufacturing, they call this “genchi gembutsu” – “go and see for yourself.”

It's really easy to see yourself as the next Steve Jobs and assume everyone's going to love what you're building. This is wishful thinking. You need to do a reality check. **Leave the building and talk to real people.**

In the worst case, you find out you were exactly on the right track, and you can build with renewed confidence. In the best case, you discover you were totally wrong about what your target customer wanted and switch to a new product strategy.

Anecdote: Toyota Sienna

The Toyota Sienna minivan's biggest market is North America. The chief engineer in charge of the 2004 Sienna, Yuji Yokoya, knew woefully little about American habits and problems. In response, he proposed a national road trip spanning 53,000 miles, covering all fifty US states, all thirteen provinces of Canada, and all states of Mexico. He drove a Sienna and talked to real Toyota customers.

Through this trip, Yokoya found that making kids happy was vital. Even though parents pay for the car, the kids are the most critical customers, particularly on long road trips. As a result, Yokoya revamped the car interior, raising its comfort and convenience levels.

If Yokoya can make a trans-Atlantic, 53,000 mile trip to learn more about his customers, you can pick up the phone and set up coffee meetings.

(Shortform suggestions: here's more advice on how to get the most out of your customer interviews:

- Focus on finding early adopters. These are the people who face the biggest pain, who are most likely to use your product first, and who will love you most when you solve their problems. If you interview a later adopter who just doesn't have the burning problem, you may receive too much discouraging negative feedback.
- **Focus on clarifying their problem, not testing your solution.** There are two reasons. 1) People often articulate their problems better than they can envision the solutions. They know what's painful, but they aren't sure exactly how to fix it. 2) You need to be open to the possibility that you

misunderstand the customer problem. Almost certainly there will be nuances to the problem you hadn't previously appreciated. If you focus too much on testing your solution, you'll be too tempted to fit a square peg in a round hole and ignore the underlying problem that might be better to fix.

- **Keep your interview procedure the same for every interview.** The key to statistics is collecting enough data points to make good inferences. If one person says your idea is terrible, treat that as just one person and collect more data points without modifying your procedure.)

If you collect sufficient data to feel like you need to change your idea, it's worth iterating through Build-Measure-Learn a few times at this stage, even without a product. Treat your interview as what you're building, and modify your interview procedure based on how the feedback is pointing you.

Then, after you've talked to potential customers and feel just somewhat convinced that the problem you want to solve exists, it's time to build and test. You will never have enough data to know for sure that people will want what you build, but you should have some real data to give you confidence that people really want what you need.

Chapter 6: Test with the Minimum Viable Product

Your goal is to move through the Build-Measure-Learn loop as quickly as possible. Even though the loop has 3 steps, Build is often the step where you will waste the most time.

The critical question you need to answer is: what is the **MINIMUM product you can build to get reliable data on your hypothesis?**

This product is termed the Minimum Viable Product (MVP) and is one of the most important concepts in Lean Startup. This is the product that is the bare minimum to test your hypothesis. Unlike normal product development, **you are NOT aiming for product perfection** – you're merely trying to start learning as soon as possible.

The value hypothesis is key to most startups – “does anyone want what we're building?” The typical wrong route many entrepreneurs make is to simply build their full product, then wait for results. Here's the problem with that approach – **every extra feature you add before testing means wasted time**. You might be pointed entirely in the wrong direction. Building features nobody wants means more wasted time before you test and find out you're in the wrong direction.

Depending on your business, there are a variety of ways to test your hypothesis in the cheapest, fastest way possible. In the more extreme, you don't need a product at all - you can simply pretend you have a product and collect user behavior.

We'll cover a range of such possibilities, from landing pages and videos to manually-serviced technology.

The Landing Page MVP

Let's say you wanted to build a new social app where you take a picture of yourself, and the app swaps your face with celebrities. You can then share the hilarious results on social media. Call it CelebrityFaceSwap.

Based on your business model research, you have some assumptions around conversion rates to make the app sustainable – out of 100 people who visit your site, 20 people might sign up, 10 people will stick around for a week, and so on.

Let's focus on the hypothesis of the 20% sign up rate (20 signups out of 100 visitors). What is the MVP in this situation? Think about it for a second.

A common answer might be: “a simple prototype that's light in features – instead of all celebrities, you can only swap faces with Kanye West. And you can only post to one social network, Facebook. We'll market it and track conversion rate for users who land on our page.”

This is much better than building a fully fledged product over the course of a year. But you can go simpler.

You actually don't even need to have a working, functional app!

Here's how. Picture a web page that describes the features of your face swapping app. You show mockups of what it looks like when your face is on Kim Kardashian's body, or when Donald Trump's face is on your body. At the bottom of the page, you have a Download button. You track clicks on this Download button.

That's all you need. You can test your hypothesis without an app at all. If you funnel in 100 users, you

can see how many people enter the page, and how many people click the button. You can tell very quickly whether you're far off from the 20% hypothesis.

If your idea is a hit, you might get great results – say, a 40% click rate. This gives you confidence to move forward.

But this number might also turn out to be 1%. In this case, a critical assumption you had about your business falls through. If 99% of people don't even bother to click Download, chances are that people don't want what you're building.

And you were able to figure this out without actually building an app! Armed with this data, you can switch your direction quickly, swapping new features into your landing page, or proposing a new product altogether.

If this is a new concept to you, it should be mind-blowing right now. **To test whether customers actually want your product, you don't actually need the product.**

Here are a few more examples of MVPs.

The Video MVP

Before building a functional product, you can simply **make a video simulating what the product does**. Even without using the actual product, watching the video will give enough info for viewers to decide whether they want the product shown.

This is what Dropbox did at its beginning. If you haven't heard of it, Dropbox is a file syncing software that synchronizes your files across your devices. If you upload a Word doc to your Dropbox folder on your laptop, you can then access the same doc on your work computer or your phone.

This is a super-complicated product to build, and also hard to describe in words. In 2007, Dropbox's founder, Drew Houston, had built an early prototype, but it wasn't fully functioning, and it wasn't ready to handle a massive number of users. But Drew still wanted to test whether people wanted his product.

So he recorded a video of his prototype, demonstrating its main features and use cases. At the bottom of the video was a form to join the waiting list for the private beta.

Drew posted this video to online communities Hacker News and Digg, and the response hit a nerve. It drew hundreds of thousands of views and thousands of people joined their beta waiting list. Without having to release his product, Drew was able to prove that people truly faced a file sync problem, and Dropbox was the solution. He could now move forward and build the product with confidence.

Concierge MVP

Many successful businesses have optimized processes to service at scale. But at your early stage, premature optimization is a huge waste of time. You don't need to handle 100,000 customers right now. You literally need to make just 1 customer happy.

With the Concierge MVP, you give each early customer the white-glove treatment. You deliver the service personally without worrying about how to scale your time. This lets you figure out whether your solution is anywhere near the right direction, without spending the hard work building efficient processes.

Let's say you want to build the meal service Blue Apron from scratch. Blue Apron sends customers pre-portioned recipes in boxes and minimizes prep time – you only need to cook the ingredients.

At scale, Blue Apron is complicated – you need warehouses to store food, assembly lines to chop and package food, and delivery services. But again, at the start, you only need to make one customer happy at first.

Your Concierge MVP would be bare-bones and manually serviced. First, you ask a potential customer what recipes she wants to prepare for dinner. You then go to the grocery store yourself, buy the ingredients, pre-package them, and give them to her in a box. You give a true concierge treatment – you follow up to learn how the cooking went, and you interview her in detail to see what you can improve about your service.

Clearly this isn't scalable. But you more than compensate with the **speed with which you can validate your hypothesis** – whether people want your product.

And the Concierge MVP has a special perk - because you're already spending so much time with your customers, you have an open door to conducting detailed customer interviews. You might find that people don't want pre-portioned food to cook at all – they just want pre-cooked food like delivery. Or you find that they get bored of recipes really quickly.

Think about it this way – if you can't make your product work with full human attention, your scalable product wouldn't work either. In many products and services, it's unlikely that an automated solution would do a significantly better job than a smart capable human could.

But if your product *does* work, you can march forward with confidence, converting what the human was doing into an automated, scalable technique.

Wizard of Oz MVP

Many startup ideas revolve around fancy automated services, especially those involving machine learning/artificial intelligence. In the Wizard of Oz MVP, you don't build the technology at all - you simply have a human perform what the machine will eventually do.

Let's use an example of an AI gift recommender. You, the customer, write a few sentences about what the gift recipient's personality and interests are. The product returns 3 Amazon links with products the recipient is likely to enjoy.

This is a pretty complicated product to build. Once again, **you don't want to waste time building the algorithms before figuring out if anyone even wants your service**. It might turn out that this concept just doesn't resonate with people, or that your recommendation strategy is poor.

So how do you test this concept? Once again, you can replace the service with a human. In a Wizard of Oz MVP, the user thinks she's interacting with an automated product, but in reality behind the curtain a human is pulling the levers. In the case of the AI gift recommender, a user could submit a few sentences on a web form. Behind the curtain, a human looks at the submission, curates a list of suggestions, and emails them to the user, pretending to be an automated service.

This gives the user a faithful representation of what the fully fledged product would act like, and you can gauge their reaction. You save a ton of product development time by using a human, who is quickly adaptable to new situations and quick to train.

This MVP strategy works because, frankly, **the user doesn't care whether the backend is powered by an algorithm or a human**. She just cares that her problem is being solved – in this case, that the gift recommendations are good and your friend's going to be happy. If customers love when a human solve their problems, they'll likely love when a machine solves them the same way.

What the MVP is NOT

Note that:

- **The MVP is NOT the smallest product imaginable.** It needs to be *enough to test your hypothesis*. Sometimes this can actually mean a functional product. Sometimes it can just be a web page. It really depends on your hypothesis. But if your MVP seems complex, consider simplifying your hypothesis to focus on the core assumption.
- **The MVP is NOT an excuse to ignore all quality problems.** Some quality issues really make a difference to the customer. Let's say you build an engaging video like Dropbox's example above, but the video takes 10 seconds to load. In response, users bounce away from the page without ever hearing about your product. If you ignore this error, you'll infer the wrong conclusion – that users don't want your product – when in fact they never learned what your product does in the first place. You need to design your MVP to make sure it reliably tests your hypothesis.
- **The MVP is NOT an excuse to "just launch and see what happens."** Your MVP should be tied to direct quantifiable hypotheses, like "10% of all visitors will sign up for the beta," or "1% of all active users will pay at least \$5." If you don't have these hypotheses set in stone, 1) you'll design a faulty MVP that doesn't lead to productive learning, 2) as you collect data, you'll fall into complacency ("well, only 2% of visitors signed up, but that's still pretty good.")
- **The MVP is NOT a magical product that you have no hope of building.** With all of these MVPs, you should be at least *somewhat* confident that you can build what's being promised. If you boast about a breathing device that lets you stay underwater for 30 minutes without scuba gear, you might get a lot of signups. But if you can't deliver, your company will fail in the long term.

You Will Likely Overbuild Your MVP

When you build your MVP, you will likely be tempted to overbuild. Your pride as a professional inhibits you from releasing low quality work. You fantasize about a world where your product is commonplace, and you can't bear to build something that's just 2% of that final vision.

But keep this in mind: **If you do not know who the customer is, you do not know what quality is.**

In other words, you may spend hundreds of hours polishing a high-quality feature that no one wants. If no one wants it, can you really say it's high quality, or that it even matters?

In contrast, if you produce a barebones MVP and customers tell you something is low quality, then that's valuable data on what you should focus on next. Your early adopters care primarily about the core of what you're offering; without bells and whistles, a promising idea will still show great metrics.

Why You Will Launch Your MVP Too Late

Finally, you're likely to launch an MVP too late because you suffer from common fallacies. Here they are,

and here's how to deal with them:

- **You're afraid that a competitor will steal your idea.** In reality, there are too many good ideas in the world and not enough good people or time to execute on them. Just *try* to pitch a product manager or venture capitalist to work on your idea – it's not going to happen. They have an abundance of ideas to work on and not enough time. You'll more likely find early on that no customers actually want what you're offering.
- **You're afraid that you'll lose your "first mover advantage" if you go public earlier.** In reality, if you make it big, competition will come after you, no matter what. And if you aren't able to fend them off in the long run, you're dead anyway. Your only long-term winning strategy is to learn faster and be better than the competition, and waiting doesn't give you any advantage.
- Ego prevents you from confronting the risk of failure. So you keep building, pushing off negative feedback so you don't feel the sting of rejection from your target users.
- **You're afraid that a minimally-featured product doesn't showcase the full potential of your vision,** and customers will reject the MVP when they would otherwise have loved your product. Look at it this way – if customers don't respond well to your MVP, you can figure out why. If it's because the MVP's missing a critical feature that was part of your vision, you can build it in and re-test. You don't lose much. But if you find that a core part of your vision is mistaken, your MVP will help you find out earlier. You have much to gain and less to lose.
- If you do fear that a prematurely negative result will make you lose hope, you can promise ahead of time that no matter what happens in the MVP, you will not give up hope until a certain number of iterations or time point. Successful entrepreneurs don't give up easily.
- **You're afraid of a bad reputation from a bad MVP that will cripple your brand.** In reality, your MVP is unlikely to gain so much attention that this is going to matter in the long run. But if you're worried about it, just launch the MVP under a different brand. You'll still be able to test and learn, without any negative consequences transferring over.
- **You're afraid of legal intellectual property or patent risk,** since showing your product to the public can complicate patenting your idea. This is a legitimate concern for some companies, and you should consult a lawyer.

As LinkedIn founder Reid Hoffman says, **"if you're not embarrassed by the first version of your product, you've launched too late."**

Be bold, and savor learning. Don't be afraid about being wrong or embarrassing yourself. You will thank yourself later.

Exercise: Design Your MVP

Try to design the minimum product that will test your hypothesis.

What is the key hypothesis you want to test? Often, it's centered around the question of whether your users will want what you're building, or whether they have the problem you're solving.

What is the Minimum Viable Product to test your hypothesis? Write a general description.

Now force yourself to simplify this – what can you throw out until you get to the *absolute* essential minimal product needed? Consider testing it without even building the product.

Chapter 7: Measure

The job of a startup is to:

1. Build an MVP to measure where it is now
2. Experiment to improve the metrics
3. Decide whether to continue in the same direction or pivot in a new direction.

Defining the right metrics that actually matter to your business is critical. Before giving examples of good metrics, we'll define common bad metrics startups choose.

Avoiding Vanity Metrics

The most insidious kind of metrics, vanity metrics, give you false optimism – it *seems* like you're making great progress, but in reality you're actually stuck.

Often, **vanity metrics are metrics that have no choice but to keep increasing over time**. One common example is total user count. Let's say your app adds 1,000 users every week. At the end of 10 weeks of hard work, you have 10,000 users. This is a big number!

Except you aren't growing any faster – you're still adding 1,000 users every week. Your hard work actually didn't change your *growth rate*.

These vanity metrics are misleading because they keep increasing over time, making you feel good. To give an obviously silly example, think about a metric like number of hours worked. Every day, you and your team each add 10 hours to the total count. At the end of the month, you proudly announce to the team, "this month, we added 1,250 hours to our Total Hours Worked metric. This is great progress. Good job, team!"

This may sound silly, but think about how many founders you've heard brag about how many hours they work. What really matters are the *results* they achieve.

Another type of vanity metric **tracks results that aren't actually critical to business function**. For instance, early startups worry about racking up press mentions or making new hires. While these may help the business move, they're not the actual core of the business – your company doesn't exist to hire employees, it exists to create value for customers and create profits for shareholders.

These common vanity metrics are all problematic:

- Total number of anything – users, sales, actions in product
- Money raised from investors
- Press articles written about your company
- Number of employees hired
- Number of features added to product
- Meetings scheduled
- Emails written

Cohort Analysis

Instead of looking at cumulative vanity metrics over time, **the more accurate analysis is to separate users**

into groups based on the time they joined, then measure your metric for each group independently . Each group of users is called a **cohort** .

For example, say we wanted to measure engagement in an app by number of photos sent. Each week, we take all the users who joined that week, and then look at the average number of photos each user sends in their first day. We work really hard for 4 weeks, and we hope to see this number rise. Instead we see this:

	Number of photos sent per user	Vanity metric – total photos sent
Week 1	5	100
Week 2	5	200
Week 3	5	350
Week 4	5	500

Notice how despite our hard work, the number of photos each new user sends has not risen – we haven't made any real progress per cohort. However, the vanity metric on the right – total photos – has risen, due to the cumulative actions of all users.

In contrast, here's what progress would look like:

	Number of photos sent per user	Vanity metric – total photos sent
Week 1	5	100
Week 2	6	250
Week 3	8	400
Week 4	10	600

The key metric of photos sent per user is rising across cohorts – hard work is paying off and actually improving the metric! (Improvement in this metric also improves the vanity metric of total photos sent, since users are sending more photos.)

Choosing Good Metrics

You need to define metrics that really matter to your business, and measure improvements to that metric.

The metrics that really matter vary from business to business. Often, they reflect your Value Hypothesis and

your Growth Hypothesis.

For example, an app might aim for these metrics:

Value Hypothesis : Each new user will upload an average of 10 photos in the first week of using the app.

Growth Hypothesis : Every user who joins the app will refer 10 new visitors, 1 of whom will join as a new user (leading to virality).

The important part is that you need to believe these numbers are vital to the success of your business. Eric Ries suggests **choosing the riskiest assumptions first** – the metrics that you have the least confidence in, yet have the most impact to your business. For example, if your company is going to be supported by advertising, advertising rates are probably not the riskiest assumption – getting user engagement will be.

Here are examples of good metrics that reveal the health of your business (only some will apply to your startup):

- Engagement
 - Time in product per user per week
 - % of users who return in the first day / first week / first month
- Growth
 - Viral factor
 - Net Promoter Score
 - Conversion rate of each major step – visiting to signup to payment
 - New users gained per week
- Finances
 - Customer acquisition cost
 - Lifetime value per user

Unlike vanity metrics, these metrics don't improve unconditionally over time – if you don't put in work, it's unlikely that cohort metrics like revenue per user or engagement will increase.

A/B Testing

(aka Split Testing)

Let's say you develop a new feature to your product and you release it to all your users. Suddenly your metrics improve. But how do you know seasonal effects aren't at play – that the users who joined later aren't just naturally more engaged? Or that you got a burst of users from an unexpected news article?

An A/B test avoids bias by splitting users into seeing two different versions of your product. By analyzing the metric resulting from both groups, you get strong quantitative evidence about which version users like more.

For example, let's say you have a landing page MVP listing your potential features and a signup form. You're not sure which of two features your users will like more. So you set up an A/B test – half of visitors see feature A on your landing page; the other half sees feature B. You measure the difference in signup rate. If feature A gets a 5% signup rate, but feature B gets 2%, this is evidence that your users may prefer feature A!

A/B testing should be applied at critical stages of your progress. You can test variations in your marketing pages, different signup and payment processes, different product interfaces, even turn features on and off in your product.

One major benefit to A/B testing is **reducing time confounds**. A common way to test is to expose all users in one week to page A, then users in the next week to page B. The issue with this is the users might not be similar - they could have come from different sources, or they might behave differently in that particular week for some reason (e.g. they're on spring break). In A/B testing, since the same group of users are randomly assigned to two groups at once, you don't need to worry that an earlier group of users is different from a later group.

Another benefit of A/B testing is that **it lowers politics**. You don't have to squabble with your team over which features are better - you can put it to the test with an MVP. A/B testing also lets you **assign credit where it's due**. If you launch a bunch of marketing and product changes at once, and your metrics improve, who's responsible? Without independent experiments, it's difficult to tell. But by running separate A/B tests, you would be able to see that marketing's redesigned pages caused the signup boost, not the new product feature.

Finally, A/B testing lets you gauge the real effect your work is having on users. The product team may obsess over features that users absolutely don't care about. A/B testing gets them to focus on results that actually make a difference.

Useful Reports

To make A/B testing, you need to have discipline around analyzing your experiments and planning the next step. Often this means producing reports, which should fit 3 A's:

Actionable : correctly designed experiments will show a clear cause and effect, and the right metrics will show whether you're really making progress. From here, you can iterate through the Built-Measure-Learn loop.

Accessible : simplify the metrics and help people understand what they mean and why they're important. Consider making metrics publicly viewable by any member of your team.

Auditable : people should be able to dig into the raw data and trace how the metrics are compiled. If someone doesn't like the result of an experiment, she may be tempted to question technicalities of the data. You need to be able to prove that the analysis is faultless.

Exercise: Choose the Right Metrics

Don't be fooled by vanity metrics - focus on the most valuable metrics for your startup.

Have you ever been asked to focus on a metric that you thought was misguided or silly? What was the metric, and why was it bad?

What metrics are you currently using to gauge the success of your startup? Could any of them be vanity metrics?

Chapter 8: Pivot or Persevere

Now when you're facing the data, you need to decide what to do. Is there still promise in your direction, and should you keep trying to iterate through the Build-Measure-Learn loop? Or have the metrics come back so disappointing so often that it's time to change your strategy entirely – to pivot?

The answer is often unclear because you will seldom encounter complete, abject failure. The more likely state is when you're barely limping along, not plummeting to the earth but also feeling like you're not really making progress. The decision gets hard here.

There are two signs of the need to pivot:

- Your metrics are not good enough to meet your goals for your startup.
- Your experiments are leading to less and less progress (which is a sign that you're out of good ideas)

Startup Example: Votizen

Lean Startup gives the example of Votizen, a company founded to boost participation in politics. Their first product was a social network where voters could unite around causes and mobilize action. They formed hypotheses around 4 metrics: signing up (registration), verifying voters (activation), sticking around and using Votizen (retention), and recruiting friends to join (referral).

Votizen built an MVP in 3 months and got their baseline metrics. Then they spent 2 more months and \$5,000 to get a second set of metrics. Then they spent another 8 months and \$20,000 to get a third set of metrics. Let's take a look:

	MVP (3 months, \$1,200)	Optimization 1 (2 months, \$5,000)	Optimization 2 (8 months, \$20,000)
Registration	5%	17%	17%
Activation	17%	90%	90%
Retention	Too low to measure	5%	8%
Referral	Too low to measure	4%	6%

Starting with just the MVP, Votizen was at a good starting point, and without a chance to improve the metrics, it was too early to pivot. The first round of optimization led to major improvements in every single metric. But the second period of optimization, costing much more time and money, led to barely a bump in metrics.

This is a sign to pivot – they had taken the current idea as far as they could go.

From user interviews, Votizen got another idea – pivot to a way to let voters contact their elected representatives easily. Users could write an email or tweet, and Votizen would print a paper letter and send

it to their Congressional representative or Senator. Even better, they could charge for this service and start funding the company through revenue.

They called this @2gov, and spent another 4 months and \$30,000 to build the prototype. Here were the metrics:

	MVP (3 months, \$1,200)	Optimization 1 (2 months, \$5,000)	Optimization 2 (8 months, \$20,000)	@2Gov Pivot MVP (4 months, \$30,000)
Registration	5%	17%	17%	42%
Activation	17%	90%	90%	83%
Retention	Too low to measure	5%	8%	21%
Referral	Too low to measure	4%	6%	54%
Payment	N/A	N/A	N/A	1%

Fantastic! Even though their new product was just an MVP, they still showed huge improvements over the most optimized version of their last product. They still had a lot of room to go to become a sustainable business, but they knew they were now on a better track.

Pivot or Persevere Meeting

To make sure your startup is on the right track, have a regular “pivot or persevere” meeting. The frequency should be between once every few weeks and once every few months, depending on your startup - the more volatile your startup and the less time you have, the more frequent this should be.

Pivot or Persevere meetings should involve product development and business leadership teams. The product team must report on its product optimization efforts over time, and the business team must report on their conversations with customers.

Why You'll Pivot Too Late

Because your startup has limited money and time, pivots are necessary to find a new direction when you're failing. **Your startup's runway is the number of pivots it can make** – so it's in your interest to get to the pivot point faster.

Yet many entrepreneurs who successfully pivoted wish they had done so earlier. Why is it so hard to pivot? It's mainly about mindset and psychology:

- Vanity metrics delude entrepreneurs on how much progress they're making, giving false confidence to keep going.
- Startups lack clear hypotheses about what metrics they need to hit. This causes a “launch and see” approach that leads to complacency around whatever results you get. Eventually you realize that the business metrics aren't enough to meet your goals, but in the meantime the metrics seem promising enough.
- Entrepreneurs are afraid to reject an idea before it had a real chance to prove itself. It's tempting to believe that the next iteration will be the major breakthrough that will turn the tides.

And yet the alternative to not pivoting is much worse – you run out of money and your company goes bankrupt.

Instead, **you need to define what failure is in the context of your startup.** Then, when you hit this definition, you need to pivot.

Types of Pivots

Pivots come in multiple flavors. Which type you make depends on what you learn in your experiments about customer needs. (Shortform note: The book doesn't give examples for these, so we've provided modern startup examples here.)

Zoom-in Pivot

Customers like a specific feature of your product. You pivot so your product now focuses entirely on delivering this feature.

Shortform Example : Justin.tv was a general live-streaming site, where anyone could stream whatever they were doing to an audience. They found that the most passionate users on their product were gamers streaming themselves playing videogames. Justin.tv pivoted to Twitch, a live-streaming site focused entirely on gaming.

Zoom-out Pivot

Your customers aren't satisfied with the bare features in your product. You pivot to expand the scope of your product and add many more features.

Customer Segment Pivot

You have the right product, but you're solving it for the wrong people. You pivot to target this new type of customer. This can happen when you deplete your early adopter userbase and expand to mainstream users, who have different preferences.

Shortform Example: Foursquare is an app that lets users check into locations like restaurants and leave reviews. It was popular in the early 2010's but fell out of favor with consumers. It [pivoted to cater to enterprise customers and developers](#) , who can use the trove of check-in data to run business analytics or power their own apps.

Customer Need Pivot

You know your customers well, and the problem you're solving for them is not very important. You pivot to target a new customer need, which may entail an entirely new product.

Example: Potbelly Sandwich Shop started as an antique furniture store that sold sandwiches to generate traffic. It realized that more people wanted their sandwiches than their furniture.

Shortform Example: Odeo was a podcasting platform company. As Odeo was failing, the staff held a last-ditch hackathon, where Twitter caught fire and spun out into its own company.

Platform Pivot

This can work in two directions. You have an application, but you realize you'd rather become a platform on which others can build their own products and applications. Or, you have a platform, but you realize you'd rather create a dedicated application that solves the end-user's needs directly.

Shortform Example: Knewton was an education technology company building test prep programs that customized to the student. It pivoted to launch a general customization platform for schools developing their own online courses.

Business Architecture Pivot

You pivot to change between two types of businesses: high margin and low volume, or low margin and high volume. This roughly matches to a business to business (B2B) model, or a business to consumer (B2C) model.

Shortform Example: In the 1970s IBM was the leader in enterprise mainframe computers, but the personal computer began disrupting the industry. After failing to dominate the PC market, IBM pivoted from hardware to provide software and consulting services, including the Watson AI platform.

Value Capture Pivot

You pivot to make money a different way. For example, you realize that instead of selling your product, you can offer it for free and make money on ads or partnerships.

Shortform Example: Duolingo, a language learning app, was previously entirely free and made money through offering crowdsourced translation services. It [introduced a subscription service](#) that removes ads and allows offline use.

Engine of Growth Pivot

You pivot to change how you grow – through virality, engagement, or paying to get users. Commonly, changing the engine of growth also requires a change in the way you capture value - for instance, going from a viral strategy to a paid marketing strategy may necessitate that you go from a free model to a paid model, to fund the marketing.

Channel Pivot

A channel is how customers get your product. For example, Johnson & Johnson uses pharmacies to reach customers, and app developers use Apple and Google's app stores to reach users.

You pivot to deliver your product to users through a different channel. For example, at first you might think of building a product and listing it on Amazon to reach customers. You pivot to sell to customers directly through your own website, once you have the traction.

Shortform Example: Television shows were previously broadcast only on TV networks, which had specific properties limiting the content: advertisers were the main revenue source, so the shows couldn't be too edgy; the network had limited time slots, so shows had to cater to the widest audience possible, diluting

quality. With streaming platforms like Netflix and Amazon, show producers now have new channels to bring their content. Furthermore, because Netflix is supported by subscriptions and not advertisers, shows are not beholden to the same restrictions as in TV.

Technology Pivot

You want to solve the same problem for the same users, but you pivot to use a different technology to do so. Often, this offers better price or performance.

Shortform Example: Netflix started out by shipping DVDs in the mail. When bandwidth increased, they pivoted to offering streaming of content over the Internet.

Example: Wealthfront started as a virtual trading league for amateur investors. The idea was to identify the best traders through their virtual portfolios, and get customers to invest in those traders. After failures on both fronts, Wealthfront pivoted to build a platform for professional managers instead of amateurs. (This wasn't their last pivot, either – in a move taking place after [The Lean Startup](#) was published, Wealthfront is now a wealth manager itself, taking money directly from customers and investing it automatically through “robo-traders.”)

Pivots are a natural part of business. You will almost certainly face less promising metrics than you had envisioned, and if you aren't able to improve these, you will have to choose to pivot or die. Even when you reach success, you need to stay vigilant for changing market conditions that necessitate a pivot.

Pivots are not willy-nilly changes in direction. They are grounded in strategic hypotheses about how to build a better business, requiring target metrics and new minimum viable products.

Part 3: Accelerate | Chapter 9: Work in Smaller Batches

Part 2 is really the core of Lean Startup. You've learned about the entire Build-Measure-Learn loop. You know that you need to form hypotheses about your business and test them with MVPs. You know to avoid vanity metrics and focus on the metrics that really tell you the health of your business. You know that you will need to face the question of whether to pivot.

In the concluding section of the book, we'll learn how to accelerate through the Build-Measure-Learn loop and stay agile as you grow.

Decreasing Batch Sizes

The aim of the Build-Measure-Learn loop is to step through it - or iterate - as quickly as you can. This increases your rate of learning.

To support this, try to decrease your batch size. Don't overinvest in huge feature releases. Release less and more often, and you'll learn faster.

The Envelope Analogy

Imagine you have 100 letters to mail. Each letter needs to be signed, folded, put in an envelope, sealed, and stamped. How would you approach this?

Your intuition is likely to batch each separate step and do all 100 at once. You'll sign all 100 letters. Then you'll fold all 100 letters. Then you'll put 100 letters in 100 envelopes. And so on.

Surprisingly, Eric Ries claims this is slower than fully processing each letter at once – a small-batch method. You'd think that you get better at each individual process when you do it repetitively. But this **doesn't make up for the additional logistics needed for big batches** - to stack 100 folded papers, stack 100 filled but unsealed envelopes, and so on.

There's another benefit to small batches – **you figure out problems much earlier**. What if the envelopes are old and the glue has worn out? What if the paper's printed in the wrong size? In a large batch, you might figure this out after you've already done 300 previous steps and have to redo your work. With small batches, you figure this out almost immediately.

This concept maps directly onto building a startup. Instead of releasing a fully-featured product once a year, you could release small batches of features regularly. With smaller batches, you detect problems and measure impact earlier. Most importantly, **you might find earlier that customers don't actually want what you're building**. Would you rather find this out incrementally with 5 small batches in 5 weeks, or 1 big batch in 10 weeks?

Anecdote: Xiaomi

Xiaomi, a Chinese smartphone maker and one of the biggest in the world, is well-known for launching weekly updates to their phones' operating systems. Engineers scour user forums looking for feature requests. They're quickly implemented, tested, and rolled out to all its users, sometimes within that

week. Users then give feedback on the new release to point out bugs and suggest new features.

Contrast this approach to the monolithic, huge-batch method of Apple, where a new version of iOS is released annually, and minor updates are introduced once every few months.

In comparison to Apple, Xiaomi users feel:

- their needs are being listened to. They're actually getting stuff they requested.
- their products are getting visibly better every week, rather than every year. There's a constant sense of progress.

Anecdote: Small Batches in Medicine

In hospitals, medications need to be shipped from the pharmacy to patient floors. A common structure is once-a-day shipment, in typical large-batch thinking. It would seem that preparing all medications at once in the pharmacy, and shipping a large batch once a day, would be more efficient.

But this thinking ignores errors that arise due to large batches. During the 24 hours between shipments, patient orders may have changed, which means meds have to be trashed or sent back to the pharmacy. In turn, this means patients don't get the meds they need, and actual health complications can result. All of these errors are very costly, more than the time gained in efficiency.

Studies have shown that reducing daily batches to 4 to 6 batches a day can reduce waste by over 30%, decrease pharmacy workload, and save money. Even though it requires more delivery staff to make more trips, it saves money in the big picture: patients get matched with up-to-date medication needs, and time-sensitive medications expire less frequently.

The Vicious Cycle of Expanding Batches

Large-batch setups tend to continuously grow batches in size. Here's why.

In traditional companies, work will follow a waterfall model, where a batch of work flows sequentially from department to department. Let's say a feature needs to be built. Marketing talks to users and gives the designers a spec of feature requirements. Designers create the user interaction and interface mockups, then pass them to the engineering team. Once the engineers are done, they ship it to QA, who approves release. Like a waterfall, the work moves sequentially from one block to the next.

In theory, this maximizes individual efficiency, like the envelope stuffing example earlier. **But it ignores the problem of imperfect communication and errors.** For example, imagine the designers pass the batch to engineering. Engineers have questions about how the UI is supposed to work, or there might be conflicts with the old UI. Engineers now have to go back to the designers, who are now delayed on their new batch. This eventually grinds progress to a halt.

There's another psychological problem with big batches – **each addition is small compared to the ever-growing batch.** If your batch size were just 2 days of work, then spending a day fixing a new bug seems like a huge investment. But if the batch size has grown to 50 days of work, what's adding one more day? No one wants to jeopardize a huge release with a potentially critical bug. Thus a bloated batch gets even bigger.

This vicious cycle keeps happening until, before you know it, you're just releasing one change per year.

How can you reduce batch size?

Here are suggestions on how to reduce batch size at your startup:

- Instead of a waterfall model, promote cross-functional teams working together and communicating face-to-face. For example, you might put engineers, designers, and marketers on the same team, tasked with rolling out new features.
- Build in automated testing. When you push small changes often, you need to make sure they're not breaking anything. Automation cuts down on overhead.
- Build a real-time metrics page that shows key performance indicators like sales and signups. If any major drops happen, red flags will sound, and you can roll back the changes to figure out where the error is. This frequent monitoring will lower the hesitation to launch a new version.
- Find faster ways to prototype. In software, modern frameworks let you build apps more quickly than ever before. Marketing software like [Leadpages](#) lets you design new landing pages in your browser. In hardware, CAD and 3D-printing speed up prototyping.

Chapter 10: How Your Startup Grows

A startup is a new company designed to grow. If you're ambitious about your startup, you want more customers and more revenue sooner. When growth stagnates, it indicates a problem with your growth strategy.

Ideally, you strive for *sustainable growth* – wherein **new customers come from the actions of past customers**. This contrasts with unsustainable one-time activities, like a publicity stunt or Super Bowl ad.

How exactly your business grows depends on your industry and product. Eric defines three major engines of growth: **Sticky**, **Viral**, and **Paid**. These aren't mutually exclusive, and often businesses will use all three at once. But it's likely one of these will dominate the others.

The Sticky Engine of Growth

The Sticky engine relies on retention of customers to grow. When you acquire the user, you want the user to stick around as much as possible.

As a metaphor, think of a leaky bucket with holes. When you pour water in, water flows out the holes. If you fill the bucket with water at the same rate it's exiting the holes, the water level stays steady – no growth. This is like a startup that loses customers as fast as it's gaining them - the total number of active users isn't growing.

But if you plug up the holes, the funnel will start filling up. Analogously, if users stick around each week, you'll continue to grow the number of users.

Sticky growth is especially important in a few types of businesses:

- In a network or marketplace of connected users. Here, the value grows exponentially with the number of users on it.
 - Social networks like Facebook: If Facebook keeps new users on and get existing users to post more, their friends will find Facebook overall more valuable.
 - Marketplaces like Uber: More riders means more rides for drivers and more earnings. This kicks off a virtuous cycle where more earnings recruit more drivers, which improves the rider experience and recruits more riders
- When your customer keeps paying you
 - Merchants like Amazon: Retaining users means they continue to buy from the merchant
 - Subscription companies like Netflix or Salesforce – regular subscriptions means the longer a customer stays engaged, the larger lifetime value you get, the more value they get from your service, and the harder it is to leave

Metrics to Care About

- **Churn rate is the key metric to care about.** This is defined as the fraction of customers who fail to remain engaged with the product in a certain time period.
- **Growth rate:** defined as new customer acquisition - churn rate.
 - For example, you may grow by 50% per month, but churn 30% of users per month. On a base of 100 customers, you gain 50 users but lose 30. You end the month with 120 customers for

net 20% growth. If you keep the same growth rate, your userbase will compound over time.

- **Engagement:** can be defined as the core engagement action you care about, like logins per week, time used per week, messages sent per month, etc.
- **AVOID total number of signups.** This will always increase, but if you're churning customers, your revenue will flatline. Your metrics need to account for users who leave or stop engaging.

How to Improve Sticky Growth

- Improve the product for existing customers to make it more engaging
- Send reminders about activity and special promotions
 - Facebook pings you on major activities – someone has added you as a friend, tagged you in a photo, has a birthday today
- For users who have left, create reminders and incentives to return
 - Subscription businesses like Netflix may offer discounts to return after you've unsubscribed

How Not to Improve Sticky Growth

Do not invest in sales and marketing if you're not retaining users. If your funnel is leaky, you'll be losing customers as quickly as you're gaining them. Pouring in more users will be very expensive and lead to zero net growth. Fix the leak first.

The Viral Engine of Growth

The viral engine relies on your existing users to bring in more users directly. Picture exponential growth – one user brings on two users. Each of those users brings on two more users. Like nuclear fission, this leads to explosive growth.

Viral growth is distinct from plain word-of-mouth – in viral growth, the referral mechanism is a **necessary consequence of using the product**. A few examples:

- 1990's email service Hotmail added a line at the end of each email sent: "P.S. Get your free email at Hotmail." Each new email sent was free advertising for Hotmail, and getting more users onboard led to even more emails sent out.
- PayPal let users send money to people without Paypal accounts. The emails said something like, "John has sent you \$20!" Who's not going to sign up to claim their money?

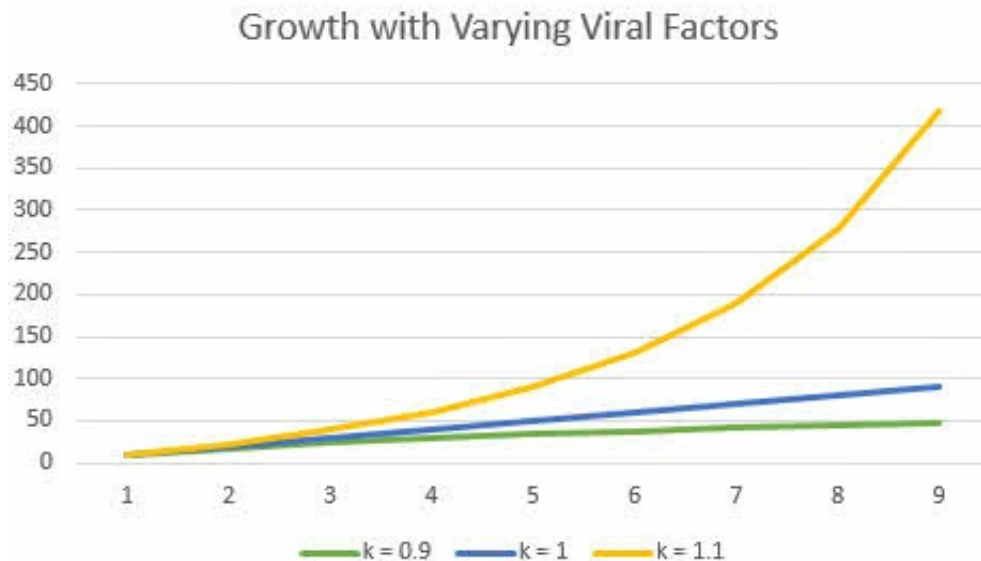
Metrics to Care About

- Viral coefficient: the number of new customers who join from invites for a single customer. For example a coefficient of 1.2 means that every new user who joins pulls in 1.2 new users on average.
 - This can be broken down further: viral coefficient = [# of invites per user] * [% of users who sign up from invites]
 - For example, if 1 user sends out 5 invites on average, and 16% of users sign up from invites, then the viral coefficient is $5 * 0.16 = 0.8$
- **A viral coefficient above 1 is viral. 1 is the magic number.**
 - If your viral factor is below 1, the invites peter out. For example, if you have a viral factor of 0.8 and start with 100 users, these 100 users recruit another 80, who recruit 64, who then recruit

51, and so forth as it sizzles down to 0. You do get a bunch of new users, but it won't explode virally.

- But if every user invites **more than 1** extra person, then it leads to exponential growth. With a viral factor of 1.1 and starting with 100 users, these 100 recruit another 110, who recruit 121, then 133, and so on.

Here's what viral growth looks like depending on what your viral factor is:



How to Improve Viral Growth

- Increase # of invites sent per user. Build in more referral mechanisms and incentivize sending more invites per user.
 - Dropbox gave early users free storage space for each new user referred
 - LinkedIn scrapes your email contacts and sends invites to people who haven't joined (an annoyingly effective feature)
 - Improving stickiness and retention also improves # of invites, since more time on your product means more opportunities to invite users
- Increase the signup rate from invites.
 - Lower the friction to signing up for your service.
 - Make your onboarding super simple so users can see the value from your service quickly.
 - Social networks like Facebook and Snapchat tend not to charge users who sign up because it would slow user growth
 - Add incentives for signing up from an invite link.
 - Many services like Uber give bonuses for both the inviter and the invitee for signing up.

How Not to Improve Viral Growth

Do not add more users to a non-viral process. If your process isn't viral, paying for more users won't make it go viral

There's an exception if your service requires a critical mass to jumpstart viral growth. But this rarely is

required, so don't deceive yourself.

The Paid Engine of Growth

This is a relatively straightforward engine of growth. Through ads or sales, you pay to acquire a user. If the user gives you more money than it cost to acquire the user, then you can use the profits to pay for more users, which gives you more profits.

Metrics to Care About

- **Customer Acquisition Cost (CAC):** How much it costs to acquire that user.
- **Lifetime Value (LTV):** How much the user gives you in value, either in direct revenue or otherwise (eg through selling ads)

Let's run through an example. An advertisement costs \$400 in total, and you get 20 new customers. Each of the customers gives you \$25 when they buy your product. This means your CAC is \$20, and your LTV is \$25. Now you have \$500 to acquire 25 new customers, which will in turn give you revenue of \$625, and so you keep growing.

How to Improve Paid Growth

The higher your profit margin, the more you can spend on more marketing.

- Increase LTV of users
 - Build a better product that creates more value, so users are willing to pay more
 - For a subscription product, get users to stay for longer
 - For ad-based products, increase engagement
 - Get users to buy add-ons and more products
- Lower cost of acquisition
 - Increase the conversion rate of users. For a fixed advertising spend, if you convert more users, you lower CAC. This can be done through:
 - Conversion optimization
 - Differentiated value among competitors. If you have a competitive advantage,
 - Find new, less competitive marketing routes

How Not to Improve Paid Growth:

Don't keep plowing money into unprofitable marketing campaigns. If you put in \$100 and get back only \$80, you'll clearly lose money fast. Try to increase the revenue or lower CAC. But also don't be afraid to abandon the channel – some just don't work for your product.

Shortform Exclusive: Avoid Paid Marketing Problems

Even though paid growth is simple in theory, you should worry about practical problems faced by many startups.

Don't lie to yourself about what your CAC is. Your CAC is not just the literal cost of acquiring the user. If you pay \$2 to get a user to signup, but that user also requires a 10-minute call by someone paid \$30/hr to signup, the CAC is actually \$7.

Be clear about your real LTV. If your LTV takes a long time to capture, don't overestimate it. Don't assume that users will love your product so much that they stick around for 24 months. Deluding yourself now will result in pain later. [Negative "unit economics" have caused many startups to fail.](#) Because modeling user value and acquisition cost can be very complex, don't deceive yourself into thinking you're profitable per user when you're not.

Keep watching the metrics. Your LTV and CAC are very likely to change over time, especially as you exhaust a marketing channel or explore new ones. As channels become unprofitable, you may have to scale them back.

Understand market competition conditions. Your competitors have likely already saturated your marketing channels and are paying at market equilibrium. If you come in and offer exactly the same product, you'll be paying the same CAC and getting the same LTVs. Over time this tends to drive toward zero profits, as competition is strong. To overcome this, you need a **competitive advantage** – a better product, lower costs, higher efficiency, etc. This allows you to pay more for a user through the same marketing channel because you get more out of the user. Out of scope, but competitive advantage covers this well.

Choosing the Right Growth Engine

In Eric Ries's experience, most startups have one engine that works especially well for them. This is particular to their product, market, and customers.

Focus on the one engine that really makes a difference for you. If it stops growing through your iteration cycles, you may need to pivot to another. But try not to maximize multiple engines at once or you'll lose focus.

None of these engines of growth will run forever. Each category of users is finite and will be depleted, and you'll have to expand to new categories of users. If you start with your early adopters, as you go mainstream, your new users' preferences will change and you'll need to adjust your product and metrics expectations.

(Shortform example: Facebook started with college students, but when it moved to high school kids and parents, it needed to change its product to encourage virality. It's a testament to their product and growth teams that they were able to take over the world – it's nowhere near as easy or pre-ordained as it seems.)

To avoid getting stuck, incubate startups within the company to provide new sources of growth. Don't start this too late, or your company may be flung into a crisis.

Chapter 11: Slow Down Intelligently

A lean startup faces natural tension between opposites: fast and scrappy vs slow and methodical, hacky and agile vs robust and overdesigned.

Initially, building a product quickly and poorly gets you data faster. But you incur technical debt that will slow development in the future.

How do you decide where on the spectrum to lie? **You need a natural feedback loop to tell you when you're moving too slowly or too quickly by identifying the root cause of problems.**

When you face a new problem, **root cause analysis** tells you precisely why the problem happened, and suggests how to fix it. This prevents overdesign and prevents the problem from happening again.

The Five Whys

Find the root cause with the **Five Whys** method. When you see a problem, ask yourself five Whys in succession.

Here's an example from Toyota where a machine broke down:

1. Why did the machine stop? There was a power overload and the fuse blew.
 1. If we stopped here, we would just replace the fuse. The fuse would blow again, and we'd replace it again, and so on. We would treat the symptom, but not the cause.
2. Why? The bearing was not lubricated.
3. Why? The lubrication pump wasn't pumping.
4. Why? The shaft of the pump was worn.
5. Why? The mechanic forgot to put a filter on, and metal scrap got in.

The final root cause is dramatically different from what you started out with. Because it's a root cause, addressing it will solve all the problems up the line. If you had stopped asking why at each stage, you'd have addressed only a symptom that will recur, instead of treating the underlying disease.

In fact, if you kept asking why a few more times, you'd probably find other root causes to tackle. The mechanic may have gotten insufficient training on this equipment. Repair of this machine might not have a checklist, which is standard practice. Or the mechanic might have just been a bad hire – in which case bad hiring practices are to blame.

This example can be applied to any recurring problem in your startup, whether it's a problem in customer service, engineering, accounting, or more.

Make a Proportional Investment

Asking Five Whys lets you figure out the root cause. **Depending on how grave the problem is, you can then make a proportional investment to fix it.**

This requires you to quantify the size of the problem. You can do this in units of resources – namely, person-hours or dollars.

A problem that occurs once and costs one man-hour to fix doesn't need a heavy process to fix it.

A problem that occurs weekly and requires ten man-hours to fix will suck up 500 hours in a year – if you can spend 100 hours to solve it completely, it's well worth it.

This calculus doesn't have to be exact – often **a rough estimate** will tell you clearly if fixing the problem is clearly a huge gain, clearly a waste of time, or somewhere on the fence.

And you can solve problems iteratively too. For first-time problems, make a smaller incremental improvement to the root cause. If the problem recurs, then you have more information about whether you want to invest more.

The Five Whys and proportional investments are a self-regulating process because **they slow you down appropriately when major problems happen**. The more problems you have, and the more severe they are, the more you invest in solutions to fix them. As better processes reduce the problems, you can speed up again. This adaptive approach helps avoid both over- and under-investment.

Avoiding the Five Blames

As powerful as the Five Whys is, it can be difficult to be objective. Sometimes a problem has multiple root causes, leading to finger pointing during a Five Whys session.

Here are a few ways to avoid the Five Blames:

- **Everyone affected by the problem should be in the room**, including the people who faced the problem on the ground and any managers involved in the decision making. This prevents the situation where the person not in the room becomes the scapegoat.
- **Be tolerant of all mistakes the first time**. Mistakes are often a result of bad systems, not bad people. If the organization has a culture of rewarding finding and fixing mistakes, people won't feel suppressed in pointing out a problem. This can be done by positive messaging rewarding the discovery of mistakes, even if the problem reporter was the one who caused the problem in the first place. Treat mistakes as growth opportunities.
- **Start with smaller, newer problems to test out the Five Whys process**. This will give you a more narrow problem, lower the emotional stakes, and limit the number of tangents and root causes. You'll end up with easier actionables to implement, and when it succeeds, it'll build confidence in the method. Don't feel tempted to tackle major, existential "baggage" problems for your startup before your team is comfortable with the Five Whys – this might derail adoption.
- **Appoint a Five Whys Master**. This person should have senior authority to assign followup work and mediate disagreements.

The Five Whys is a natural feedback loop that slows you down when you're moving too fast. By finding the root cause and fixing it, you'll prevent costly mistakes that will cripple your speed in the long run.

Exercise: Try the Five Whys

Try to figure out the root cause of a problem.

What problem have you faced recently? Why did it happen?

In turn, why did this happen?

In turn, why did this happen?

In turn, why did this happen?

Chapter 12: Startups in Big Organizations

The final substantive chapter of Lean Startup discusses innovation in a large company and internal startups.

If growth is your goal and you achieve it, you'll keep growing your startup to 10, 100, 1,000 people. How do you keep innovating to grow new lines of business while keeping existing products competitive? How do you prevent yourself from being bogged down by process?

The solution is to **manage lines of innovation in parallel**.

Each product has a life cycle:

- new product
- growth and scaling
- optimization, operational excellence, and fighting newcomers
- legacy support and cutting costs

Once a company grows past its early stages, it needs to think about what comes next. This means managing multiple products at different stages of the cycle at once. Earlier projects will be sunsetting and losing revenue just as the next major innovation breaks new ground.

Match People with the Product Stage

Who is the best person to run a product through its lifecycle? A typical answer is to tie the original product team through the entire life cycle of the product. After all, they built it, so they seem like natural candidates to scale it up and to sunset the product.

This may be sub-optimal. **People tend to be specialists at one of the phases in the life cycle**. Some people are natural early-stage innovators and don't have the energy or conscientiousness for optimization. Others like the rigor of operational excellence, but can't stomach the variability in the earliest stages.

Instead of keeping the original team with the product, **keep people in the stage they're good at. Then transfer products between teams** - once a product matures past the growth phase team, move it to the operational maintenance team. This means some people's roles may focus on spinning up new early-stage startups, and others inherit those startups and grow it to scale.

The Three Attributes of Successful Startup Teams

The author suggests that successful innovation requires three structural attributes:

- **Limited but secure resources.** By their nature, startups are high risk and thus deserve less resources than surefire investments. This is a good thing, since it forces startups to focus on the right questions or perish. But because a sudden change in resources can be catastrophic, internal startups need their funding secured and immune from tampering by other managers.
- **Independent decision-making authority.** To move faster, startups need to be able to run and execute experiments without passing each one by a review board. Building cross-functional startup teams allows representatives from each stakeholder department to partake in the innovation and sign off quickly on decisions. Of course, this independence needs to be balanced with safeguards - internal startups shouldn't do anything that can damage the entire brand or hurt customers, for example.

- **Incentive in success.** Entrepreneurs, internal or independent, are motivated by tying their personal success to their startup's success. Typically this means equity or ownership, and in an internal startup this might be bonuses or profit sharing. But the stake doesn't have to be financial – it can be public recognition and career advantages. It's important that the rewards be given fairly, or other internal entrepreneurs will lose trust in the system.

An Innovation Sandbox

It's natural for internal startups to threaten the existing parent organization. They may offer new products that cannibalize customers of the incumbent product or offer different pricing structures. The startup might even endanger the entire current business.

These fears are all reasonable, but they lead internal stakeholders to delay decisions and obfuscate data.

The common solution is to hide internal startups in a secret skunkworks team, which will work in the short-term but is counterproductive in the long run. When the innovation is finally shared with the main company managers, they'll feel deceived and resist future innovation. This leads to politics and paranoia, where people will question new projects and seek out threats to their standing.

So how do you achieve the right balance of freedom and risk control? **Create an innovation sandbox** where the startup team's impact will be insulated from the main company, but where they can have more autonomy. The sandbox has these components:

- Any team can create an A/B test of limited scope
- One team must see the experiment through from beginning to end
- The experiment is limited in calendar time
- The experiment is limited in number or % of customers affected
- Every experiment is evaluated on the same standard 5-10 actionable metrics
- The team must monitor metrics and customer reactions live. If something catastrophic happens, abort the experiment.

This structure limits the damage to the larger company, while giving startup teams autonomy to iterate quickly. If the experiment shows improved metrics in this limited scope, the innovation can expand its reach and ultimately be reintegrated into the main company.

Chapter 13: Epilogue

Eric Ries believes a tremendous amount of effort today is wasteful. Over the past century, our bandwidth for production is much larger than our ability to know the right things to build. This problem is compounded by the high rate of change of many industries.

Luckily, this is preventable. **The Lean Startup is a framework for figuring out the right things to build.** It answers the innovation question: how can we build a sustainable organization around a new product or service?

Lean Startup accomplishes this through the scientific method: forming testable hypotheses, testing them, and gathering data for important metrics. This is validated learning.

Lean Startup is NOT meant to be a rigid, codified set of practices. If you blindly follow the steps outlined here without care, you may find yourself obsessing about stepping through the motions, rather than isolating the core of what really matters to your startup. Eric does not want Lean Startup to be a checklist that people go through perfunctorily. You may not always want to launch a low-quality prototype, for example.

The important goal is validated learning, discovering the truth about the world in a rigorous way. Think about Lean Startup as a mental framework for how to think about building a sustainable business.

What is your value hypothesis or growth hypothesis? What is the fastest, cheapest way you can validate this hypothesis?